0.1 Background

Нам в этой лекции нужно будет говорить про вероятностный подход и работу с распределениями. Поэтому прежде чем переходить непосредственно к обсуждению генеративных моделей, давайте вспомним несколько полезных фактов.

Пусть у нас есть случайная величина – некоторая величина, принимающая некоторое значение при нашем случайном эксперименте. Например, всем известное бросание игрального кубика. И соответственно есть распределение случайной величины, которое сопоставляет значения случайной величины и вероятности выпадения этих значений. Распределения бывают дискретные и непрерывные, важно, что сумма всех вероятностей в распределении должна равнятся 1.

Еще важный момент, который стоит понимать это разница между функцией распределения и плотностью распределения. Функция распределения это функция, характеризующая вероятность того, что случайная величина X примет значение, меньшее x, где x — произвольное действительное число. В английской литературе ее называют Cumulative Distribution Function или CDF.

$$F(X) = \mathbb{P}[X \leq x]$$

В то время как плотность распределения это как раз функция, которая как раз сопоставляет значение случайной величины с вероятностью выпадения этого значения. Вот чаще всего в формулах, если вы видите букву p, то в ней подразумевается именно плотность распределения случайной величины или также еще ее называют функцией распределения вероятности (не путать с функцией распределения случайной величины, которую мы упомянули выше), и в задачах генеративных моделей, мы ищем как раз плотность распределения случайной величины. В английской литературе ее называют как PDF(probability density function).

В дальнейшем, говоря о распределении, имеется ввиду именно понятие плотности распределения.

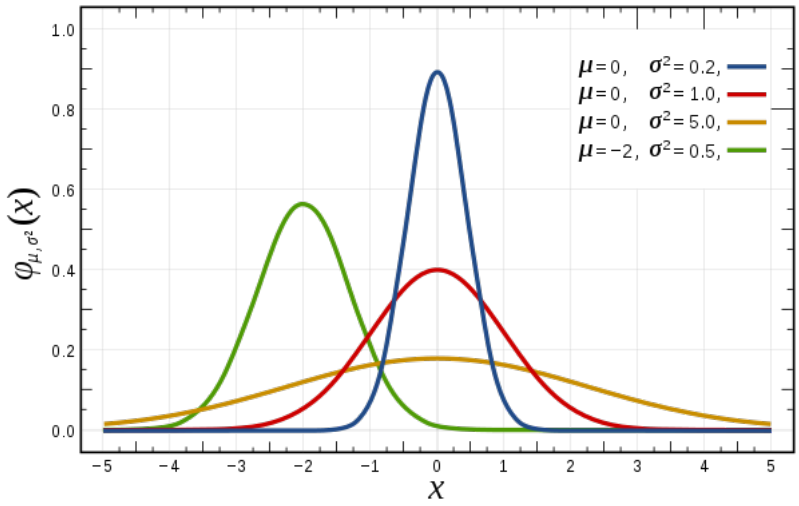


Рис. 1: Нормальные распределения с различным средним и дисперсией

У распределений вероятности есть несколько очень важных характеристик, например **матожидание**. Матожидание – среднее, взвешенное по вероятностям, значения случайной величины. Часто обозначается через букву $\mathbb{E}$ и по сути является центром тяжести распределения.

Для дискретной случайной величины X:

$$\mathbb{E}[X] = \sum_i x_i \cdot P(X = x_i) \tag{1}$$

Для непрерывной случайной величины X:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot p(x)\, dx \tag{2}$$

Важными свойствами матожидания являются:

- Линейность:

$$\mathbb{E}[aX + bY] = aE[X] + bE[Y]$$

- Математическое ожидание константы:

$$\mathbb{E}[c] = c$$

Вторая важная характеристика распределения – это его **дисперсия** – мера разброса случайной величины относительно её матожидания. Квадратный корень из дисперсии называется **среднеквадратическим отклонением**.

Для дискретной случайной величины X:

$$\mathbb{D}[X] = \sum_i (x_i - \mathbb{E}[X])^2 \cdot P(X = x_i) \tag{3}$$

Для непрерывной случайной величины X:

$$\mathbb{D}[X] = \int_{-\infty}^{\infty} (x - \mathbb{E}[X])^2 \cdot p(x)\, dx \tag{4}$$

Связь с математическим ожиданием:

$$\mathbb{D}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] \tag{5}$$

Дисперсия неотрицательна, но в отличие от матожидания – нелинейная и зависит еще от значения ковариации между величинами.

$$\mathbb{D}[X + Y] = \mathbb{D}[X] + \mathbb{D}[Y] + 2\mathrm{Cov}(X, Y)$$

$$\mathbb{D}[aX] = a^2\mathbb{D}[X]$$

По сути ковариация показывает зависимость величин и выражается следующим образом:

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] \tag{6}$$

В случае независимых случайных величин мы получаем линейную дисперсию. Давайте это докажем.

Раскроем выражение ковариации двух случайных величин:

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY - X\mathbb{E}[Y] - Y\mathbb{E}[X] + \mathbb{E}[X]\mathbb{E}[Y]], \tag{7}$$

и т.к. $\mathbb{E}[X] = const.$ т.е. это просто число, какое то среднее значение случайной величины X(например, у нормального распределения среднее значение равно 0), $\mathbb{E}[\mathbb{E}[X]] = \mathbb{E}[X]$, так как мат. ожидание числа это само число. Тут есть тонкий момент в $\mathbb{E}[X\mathbb{E}[Y]]$. Так как $\mathbb{E}[Y] = const.$, а $\mathbb{E}[\alpha X] = \alpha\mathbb{E}[X]$, то мы можем вынести $\mathbb{E}[Y]$ из под мат. ожидания, т.е. $\mathbb{E}[X\mathbb{E}[Y]] = \mathbb{E}[Y]\mathbb{E}[X]$. С учетом всех выкладок, ковариация будет равно следующему

$$\begin{aligned}
&\mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \\
&\mathbb{E}[XY - X\mathbb{E}[Y] - Y\mathbb{E}[X] + \mathbb{E}[X]\mathbb{E}[Y]] = \\
&\mathbb{E}[XY] - \mathbb{E}[X\mathbb{E}[Y]] - \mathbb{E}[Y\mathbb{E}[X]] + \mathbb{E}[\mathbb{E}[X]\mathbb{E}[Y]] = \\
&\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] - \mathbb{E}[X]\mathbb{E}[Y] + \mathbb{E}[X]\mathbb{E}[Y] = \\
&\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y],
\end{aligned} \tag{8}$$

Итак, получается ковариация равна

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y], \tag{9}$$

и для независимых случайных величин

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y] \tag{10}$$

то

$$\mathrm{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] = \mathbb{E}[X]\mathbb{E}[Y] - \mathbb{E}[X]\mathbb{E}[Y] = 0, \tag{11}$$

Отсюда и дисперсия тогда равна

$$\mathbb{D}[X + Y] = \mathbb{D}[X] + \mathbb{D}[Y] + 2\mathrm{Cov}(X, Y) = \mathbb{D}[X] + \mathbb{D}[Y]$$

т.е. для случайных независимых величин дисперсия линейна, а не квадратична.

Самое важное, что следует понять, что мат. ожидание это среднее значение случайной величины, а дисперсия это мат. ожидание квадрата отклонения этой случайной величины от ее математического ожидания, или грубо говоря, просто величина, которая показывает на сколько могут отклоняться значения случайной величины от ее среднего значения.

0.1.1 Совместное и условное распределения

Также важно отметить, что если у нас есть две случайные величины с известными плотностями распределний, нас может интересовать их совместное распределение, которое в общем случае не равно произведению плотностей, но равно в случае независимых величин. И условная плотность вероятности, вероятность X, при условии Y. То есть как знания о значении у влияет на величину x. Например, с какой вероятностью человек возьмет зонтик, при условии, что идёт дождь.

Формально, если у нас есть две случайные величины X и Y, то условное распределение X при условии $Y = y$, обозначаемое как $P(X|Y)$, определяется следующим образом:

$$P(X|Y) = \frac{\overbrace{P(X,Y)}^{\text{joint}}}{\underbrace{P(Y)}_{\text{marginal}}} \tag{12}$$

То есть условная вероятность выражается через совместное распределение (**joint**) и распределения случайной величины (**marginal**). Если величины независимы, то безусловное распределение равно условному. Также из этого распредедения следуют два основных свойства:

- **Правило произведения**

$$P(X_1, X_2, \ldots, X_n) = P(X_1) \cdot P(X_2|X_1) \cdot P(X_3|X_1, X_2) \cdot \ldots \cdot P(X_n|X_1, X_2, \ldots, X_{n-1})$$

- **Правило суммирования вероятностей**

$$P(X) = \int P(X,Y)dY = \int P(X|Y)P(Y)dY$$

0.1.2 Теорема Байеса

Далее хотелось бы отметить теорему Байеса, которая решает обратные вероятностые задачи. Человек взял сегодня зонт, с какой вероятностью на улице шёл долждь? То есть на основе наблюдений X, хотим спрогнозировать Y. Пусть мы знаем, как Y влияет на X, знаем, какие значения Y умеют принимать, то есть знаем некоторое априорное распределение. И есть какое-то знание, которое мы пронаблюдали – $P(X)$. То есть зная условное (**likelihood**), априорное (**prior**) и наблюдаемое (**evidence**) – выражаем апостериорное распределение.

$$\underbrace{P(Y|X)}_{posterior} = \frac{\overbrace{P(X|Y)}^{likelihood} \times \overbrace{P(Y)}^{prior}}{\underbrace{P(X)}_{evidence}} \tag{13}$$

Теорема Байеса позволяет обновить вероятности гипотез на основе новых данных, что делает ее мощным инструментом в статистике, машинном обучении и других областях.

0.1.3 Maximum Likelihood Estimation

Последнее, что хотелось бы обсудить в данном небольшом введении – **функция правдоподобия**. Функция правдоподобия – это функция, описывающая вероятность наблюдать конкретный набор данных при заданных параметрах модели. Пусть дана выборка $\{x_i\}_{i=1}^n$ из независимых одинаково распределенных величин из какого-то распределения $p(x|\theta)$, известного с точностью до параметра θ. Задача – оценить параметр θ данного распределения. Как можно это сделать? С помощью оценки максимального правдоподобия – максимизируем вероятность нашей выборки при условии параметра. Так как величины независимы функция правдоподобия $\mathcal{L}(\theta)$ определяется как произведение плотностей вероятности всех наблюдений:

$$\mathcal{L}(\theta) = p(x_1|\theta) \cdot p(x_2|\theta) \cdot \ldots \cdot p(x_n|\theta) \tag{14}$$

Оценка максимального правдоподобия (MLE) $\hat{\theta}_{\text{MLE}}$ – это значение параметров θ, которое максимизирует функцию правдоподобия:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} L(\theta) \tag{15}$$

Поскольку нам не очень удобно работать с произведениями из-за сложности вычислений, можно перейти к сумме логарифмов. При этом логарифм никак не изменяет монотонности и максимум останется в той же точке. Поэтому использование логарифма максимального правдоподобия упрощает процесс оценивания параметров модели и обеспечивает более стабильные и эффективные вычисления, особенно при работе с большими выборками данных и сложными моделями.

0.2 Генеративные модели

Теперь наконец предлагаем перейти к основной теме – генеративные модели. До этого вы работали с **дискриминативными моделями**. Пусть у нас есть какая-то известная выборка X, есть модель с параметрами W, которые должны настраивать в процессе обучения, а на выходе мы хотим предсказать некоторые лейблы Y. То есть мы моделируем распределение $p(Y, W|X)$, однако мы не моделируем распределение на X.

Обычно эти распределения имеют более простую структуру и важной особенностью является то, что мы не генерируем новых объектов. Как правило, это задачи:

1. классификации
2. сегментации
3. детекции

В таком случае X – это наши изображения с котиком, W – веса нашей модели, а Y – класс, маска, bounding box. При этом не обязательно это могут быть задачи компьютерного зрения. Например, задача по речи человека предсказать пол, возраст говорящего. Или же задача машинного перевода на другой язык (следует заметить, что в случае машинного перевода – наши лейблы уже не обязательно имеют более простую структуру).

Кроме дискриминативных моделей есть **генеративные модели**. Эти модели уже моделируют совместное распределение на все три группы переменных $p(Y, W, X)$. То есть можно генерировать новые объекты и нам необходимо как-то промоделировать распределение на X.

К таким задачам можно отнести генерацию картинок по тексту, которые сейчас очень популярны, генерацию видео, 3d объектов, музыки и многое другое. Это могут быть диффузионные модели, ганы или автокодировщики.

И на этом занятии наша основная цель – обсудить вариационные автокодировщики (VAE).

0.3 Автокодировщики

Для начала давайте разберёмся, что такое автокодировщик. Автокодировщик состоит из двух составляющих:

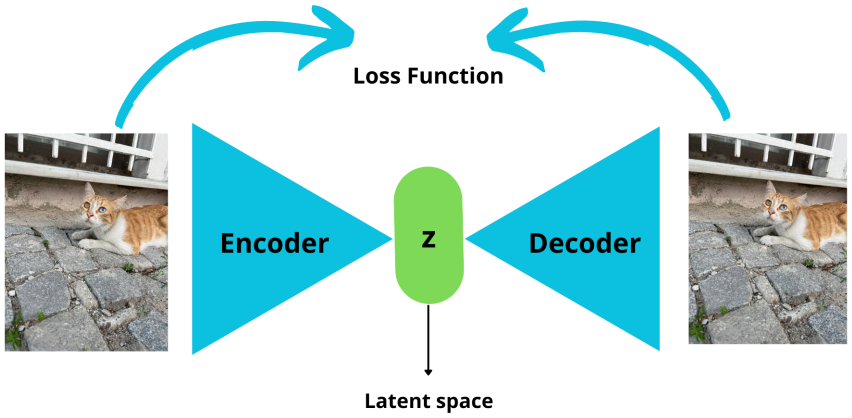


Рис. 2: Схема автокодировщика

- **Энкодер** – кодирует наши данные в некоторое латентный вектор из латентного пространства.
- **Декодер** – декодирует ланетный вектор в пространство наших данных.

Таким образом, автокодировщик пытается сопоставить исходному объекту декодированный. Важной особенностью является то, что мы не накладываем каких-то дополнительных условий на латентное пространство. Однако мы хотели бы уметь не просто кодировать/декодировать, но и генерировать объекты и не накладывая никаких условий на латентное пространство, процесс сэмплирования из него для дальнейшей генерации становится очень сложным. Также важно подобрать хорошую функцию потерь, однако для обучения чаще всего используется квадратичная функция потерь – MSE.

0.3.1 Для чего нужны автокодировщики?

Автокодировщики могут использоваться для различного пула задач. Давайте рассмотрим несколько примеров.

Уменьшение размерности

Размерность играет очень важную роль в решении различных задач машинного обучения. Очень часто имеющиеся ресурсы и доступное время не позволяет нам оперировать с данными очень высоких размерностей. В таком случае важно уметь уменьшать её, теряя при этом как можно меньше важной информации об исследуемом объекте. Одним из распространённых методов, решающих проблему уменьшения размерности, является principal component analysis (PCA). Однако данный метод проводит аппроксимацию линейными подпространствами. Автокодировщик, в то же время, является нелинейным обобщением PCA и позволяет качественно кодировать объекты в низкоразмерное пространство, а затем декодировать обратно.

Рассмотрим для примера датасет MNIST, состоящий из набора чёрно-белых изображений цифр 3.

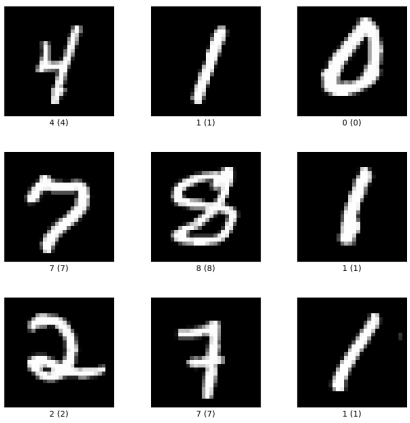


Рис. 3: MNIST dataset

Отобразим пространство наших картинок в латент размерности два. Заметим, что несмотря на то, что мы не дали каких-то дополнительных априорных знаний о латентном пространстве, модель сама научилась различать их и мы видим достаточно четко выраженные кластеры различных цифр.

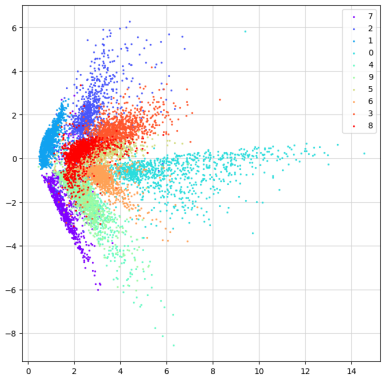


Рис. 4: Латентное пространство размерности 2 для MNIST датасета

Детекция аномалий

Также автокодировщик может использоваться для задачи детекции аномалий. Предположим мы обучили модель, которая умеет кодировать-декодировать котиков.

Наш автокодировщик видел только картики котиков в своей обучающей выборке. А теперь представим, что на вход энкодера попало изображение с собакой 5.

Поскольку наш автокодировщик не знает ничего о подобных картинках, на выходе мы получим что-то странное и неправильное. Сравнив предсказанное изображение, мы не получим картинку, близкую к исходной. Тогда, посмотрев на разность между изображениями, мы сможем определить, что данная картинка с собачкой – аномалия и дальше нам её использовать не нужно.

Рис. 5: Аномалия

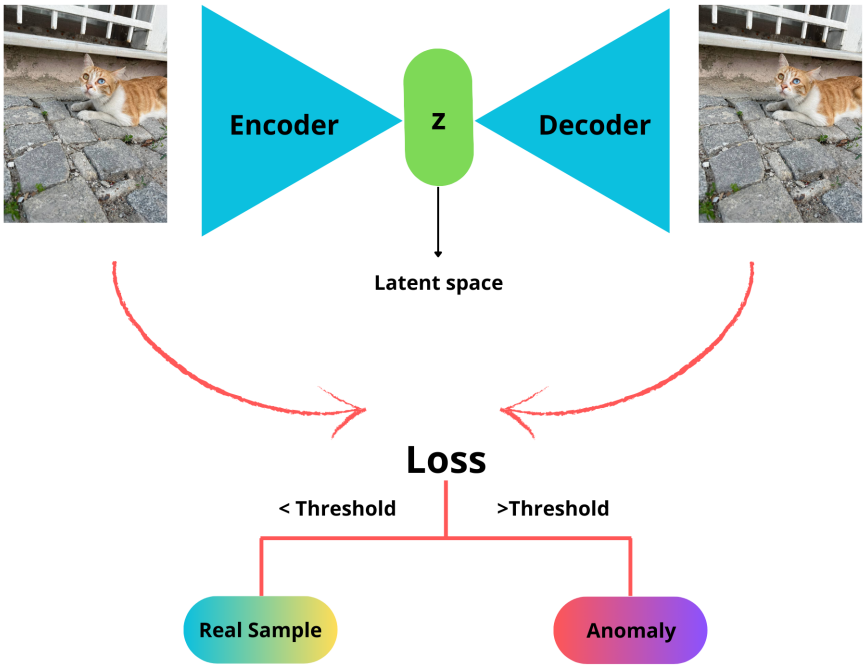


Рис. 6: Схема детекции аномалии

0.3.2 Denoising autoencoder

Предположим, что у нас имеется закоррапченное (испорченное) изображение. Мы могли добавить к нему шум, могли вырезать кусочек или наоборот добавить что-то лишнее. Например, некачественное фото, случайно вырезанная маска и так далее. Задача же наша заключается в том, чтобы получить исходное изображение.

Для этого мы можем использовать автокодировщик. Посмотрим на предложенную схему 7.

Пусть у нас имеется какое-то изображение. Добавим в него шум/маску/понизим разрешение или что-то другое, в зависимости от нашей задачи. Затем с помощью энкодера закодируем "испорченное" изображение в латент, а затем декодируем обратно. При этом функцию потерь накладываем не на "испорченное" изображение, а на исходное и предсказанное. Таким образом, автокодировщик выучивает восстанавливать изображение после определенного рода манипуляций над ним.

Как уже было сказано, можно использовать это для различных задач, например для:

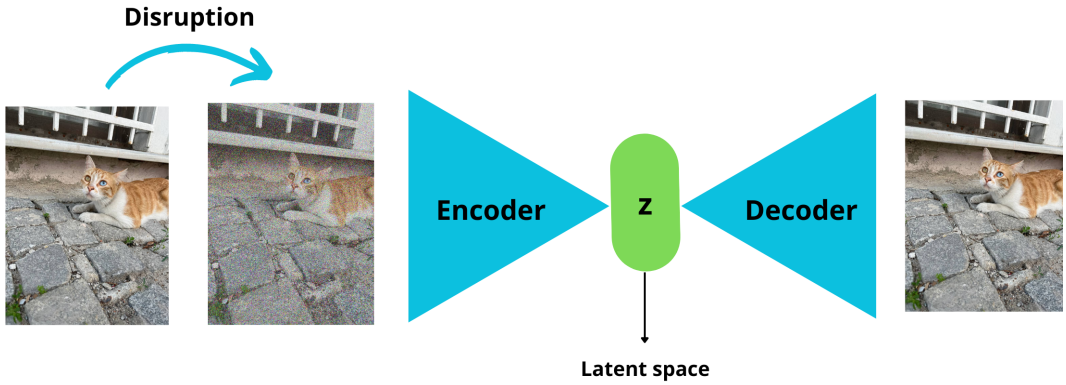


Рис. 7: Схема denoising autoencoder

- **Inpainting**

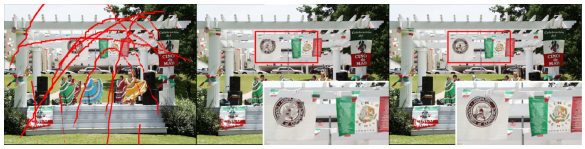


Рис. 8: Автокодировщики для inpainting [4]

- **Super Resolution (SR)**

Рис. 9: Автокодировщики для SR[4]

- **Debluring**

Рис. 10: Автокодировщики для debluring[4]

0.3.3 DeepFakes

Теперь рассмотрим не совсем стандартный пример использования автокодировщиков, а именно для задачи генерации DeepFakes. Одна из возможных идей – это менять лица на изображениях или видео. Давайте посмотрим, как можно использовать автокодировщик для данной задачи. Пусть у нас есть два человека: Джими Фэллон и Джон Оливер. Мы хотим научиться менять лицо одного на другого.

Для этого будем следовать следующему алгоритму 11:

- Обучаем общий энкодер кодировать оба типа лиц, но при этом декодеры используем разные.
- На этапе валидации используем энкодер для кодирования одного лица, но декодер второго.

Однако в таком ванильном сценарии качество получается не очень хорошее. Дополнительной важной особенностью такого подхода является аугментация изображений - некоторый коррапт, причем не просто добавление шума. Хорошо в таком случае работает метод варпинга - берём какую-то сетку с узлами и накладываем на картинку. Затем сдвигаем некоторые узлы и интерполируем сдвиги. В результате получается что-то, что вы видите на левых картинках (немного сдвинутое странное лицо). И именно такой элемент, добавленный в обучение помогает хорошо проводить подмену лиц.

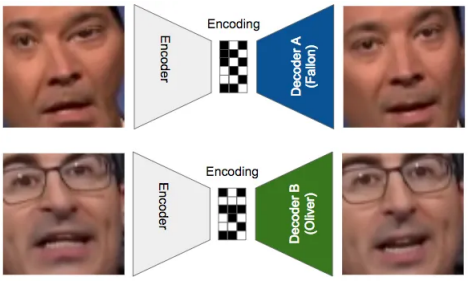


Рис. 11: Схема использования автокодировщика для DeepFakes

На выходе можно получить вполне неплохую смену лиц. Однако, конечно, сейчас технология DeepFakes шагнула далеко вперед, это пример простого использования автокодировщика.

0.4 GLO

Использовать автокодировщик для генерации изображений - довольно сложная задача. Мы не всегда представляем, как именно выглядит наше латентное пространство и как из него сэмплировать. Поэтому авторы статьи Generative Latent Optimization (GLO) [1] предлагают идею **автодекодера**. Предлагается обучать модель, которая сопоставляет изображение определенного класса с некоторым вектором на единичном гипершаре определённой размерности. То есть если наша цель - картинки с котиками, мы пытаемся сопоставить пространству изображений наших котиков - часть этого шара 12.

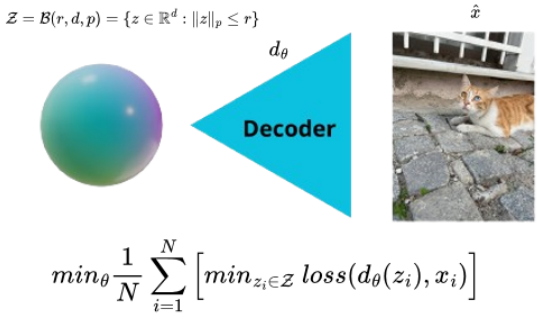


Рис. 12: GLO

Для этого мы учим не только нейросеть (декодер), но и латентное представление всех наших тренировочных примеров. То есть для каждого элемента нашей обучающей выборки x_i мы сопоставляем латентный вектор z_i, такой, что $d_\theta(z_i)$ нейросеть декодирут нам изображение близкое к исходному x_i. Функция потерь в таком случае получается из суммирования по всем обучающим примерам некоторой дистанс функции (функции, показывающей различие между исходным и сгенерированным изображением), например MSE. Оптимизация же проводится не только по параметрам нейросети, но и по латентным векторам.

Таким образом, в данном примере у нас нет энкодера, который как-то кодирует изображение, есть только декодер, поэтому такой тип модели называется **автодекодером**. Такая модель умеет интерполировать и восстанавливать тренировочные сэмплы. Реконструкция довольно хорошая, но сильно сглаженные 13.

Однако как провести генерацию с помощью такой модели, ведь чаще всего распределению исходных картинок сопоставляется лишь небольшая часть шара и чтобы получать адекватные результаты авторы предлагают следующий алгоритм. После обучения у нас есть набор тренировочных латентов. Зафиттим гауссиану в эти вектора, а затем уже сэмплируем из этого гаусса. Если посмотреть на результаты генерации, то видно, что достаточно много артефактов и в более большом разрешении изображения становятся довольно размытыми, однако в то же время качество генерации лучше чем у VAE (который мы рассмотрим чуть позднее) 14.

У такой модели есть ряд недостатков. Во-первых, она очень чувствительна к инициализации векторов латентного пространства. Во-вторых, если мы хотим добавить картинку, то для того, чтобы найти соответствующий ей вектор в латентном пространстве - необходимо провести оптимизацию. И наконец, нам необходимо сохранять все латентные вектора от картинок для дальнейшей работы с ними.

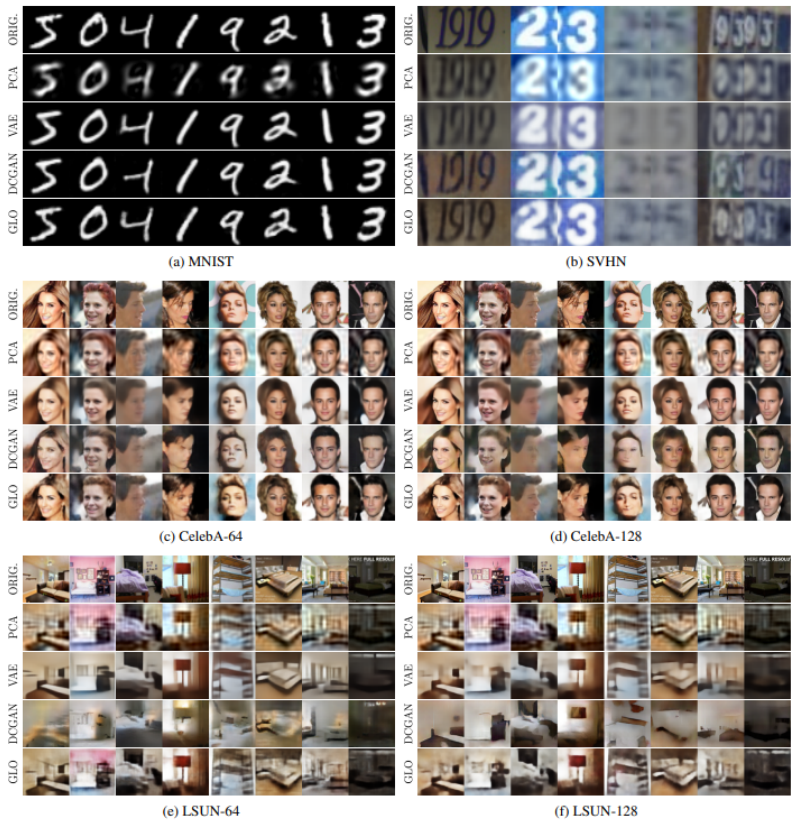


Figure 5. Reconstruction results from PCA, VAE, DCGAN and GLO on the 4 datasets. The original images are on the top row. VAE reconstructions are blurrier than GLO . DCGAN fails to reconstruct images from large datasets.

Рис. 13: GLO reconstruction, image source [1]

Figure 4. Samples generated by VAE, DCGAN and GLO on the 4 datasets. For CelebA and LSUN, we consider images of size 64 and 128. On small datasets, the three models generate images of the comparable quality. On LSUN, images from VAE and GLO are nowhere close to those from DCGAN.

Рис. 14: GLO generation, image source [1]

0.5 VAE

Как мы увидели, у автодекодера есть довольно много проблем с генерацией. Также в процессе обучения нам приходилось оптимизировать латенты для каждого элемента обучающей выборки. Чтобы избавиться от лишней оптимизации предлагается предсказывать эти векторы из исходного изображения, также как это делалось в обычном автокодировщике.

Однако нам надо еще научиться качественно сэмплировать из латентного пространства. Для этого мы предсказываем не конкретный латентный вектор, а некоторое распределение данных латентных вектором, соответствующих исходному изображению X 15. Чаще всего в качетсве такого распределения рассматривается нормальное, из которого затем сэмплируется латентный вектор, который уже можно декодировать [7].

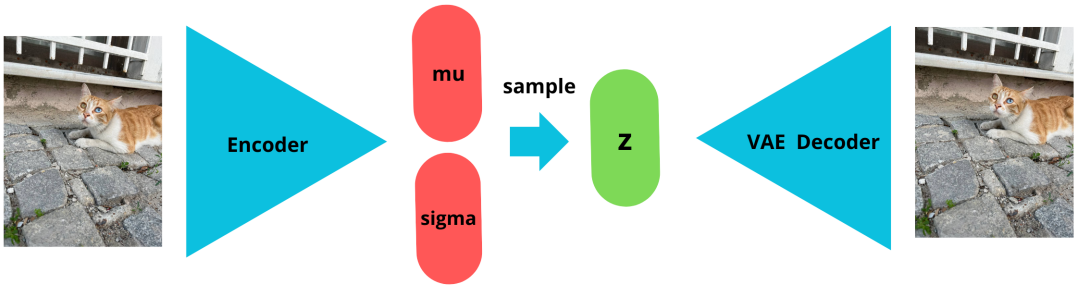


Рис. 15: VAE

Прежде чем мы начнём разбираться как такую модель обучать, давайте вспомним или узнаем, как сравнивать распределения, потому что нам это понадобится.

0.5.1 KL Дивергенция

Если мы хотим сравить два вектора - мы можем посчитать MSE loss или посмотреть их косинусное расстояние или придумать что-то еще. Но что делать, когда нам надо сравнить два распределения? Для такой цели часто используют KL-дивергенцию:

Для непрерывных распределений:

$$D_{KL}(p\|q) = \int p(x) \log \frac{p(x)}{q(x)} dx \tag{16}$$

Для дискретных распределений:

$$D_{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \tag{17}$$

Например для двух нормальных распределений 16

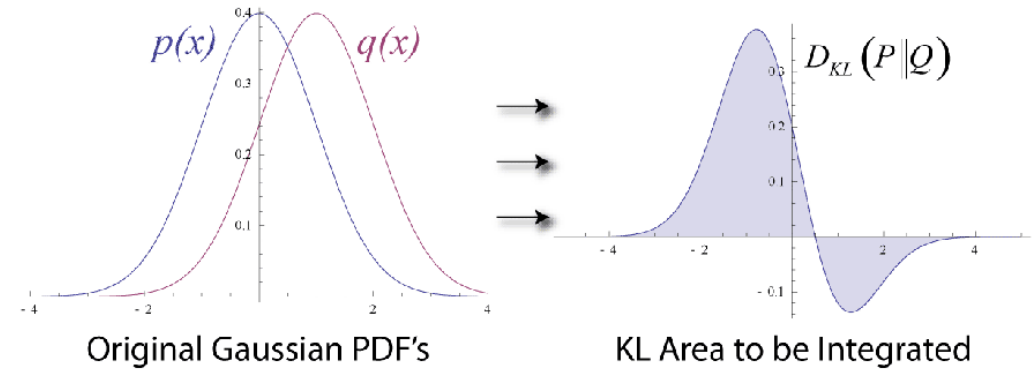


Рис. 16: KL дивергенция между двумя гауссианами

Она обладает несколькими свойствами:

1. Дивергенция одинаковых распределений равна 0 $D_{KL}(p\|p) = 0$
2. Она не отрицательна $D_{KL}(p\|q) \geq 0$
3. Дивергенция не симметрична $D_{KL}(p\|q) \neq D_{KL}(q\|p)$

Первое свойство следует сразу из определения, докажем тогда второе. Рассмотрим непрерывный случай:

$$-D_{KL}(p\|q) = \int p(x) \log \frac{q(x)}{p(x)} dx \underbrace{\leq}_{\text{Неравенство Йенсена}} \log \int p(x) \frac{q(x)}{p(x)} = 0 \rightarrow D_{KL} \geq 0 \tag{18}$$

Теперь давайте немного подробнее рассмотрим третий пункт. Пусть у нас есть два распределения $P(x)$ и $Q(x)$, при этом мы хотим минимизировать $D_{KL}(P\|Q)$. Тогда обратим внимание на рисунок 17. Заметим, что в первом случае у нас будет слишком большие значения во второй моде распределения P и значит KL-дивергенция будет большой после интегрирования. Однако во втором случае, несмотря на то, что у нас уже нет такого хорошего совпадения в одной моде - KL-дивергенция будет иметь небольшое значение.

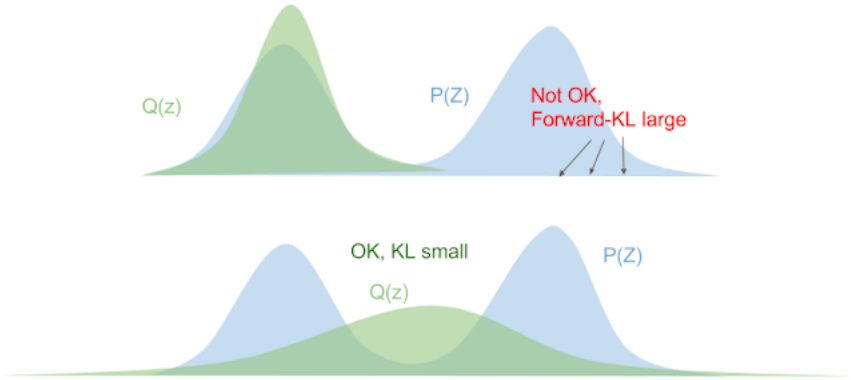


Рис. 17: KL дивергенция между P и Q

Теперь давайте рассмотрим $D_{KL}(Q\|P)$ 18. Заметим, что здесь у нас уже обратная ситуация. И есть проблема с тем, что $Q(z) \neq 0$ в то время как $P(z) = 0$, что опять же даёт очень большой вклад в интеграл и KL-дивергенция получается большой.

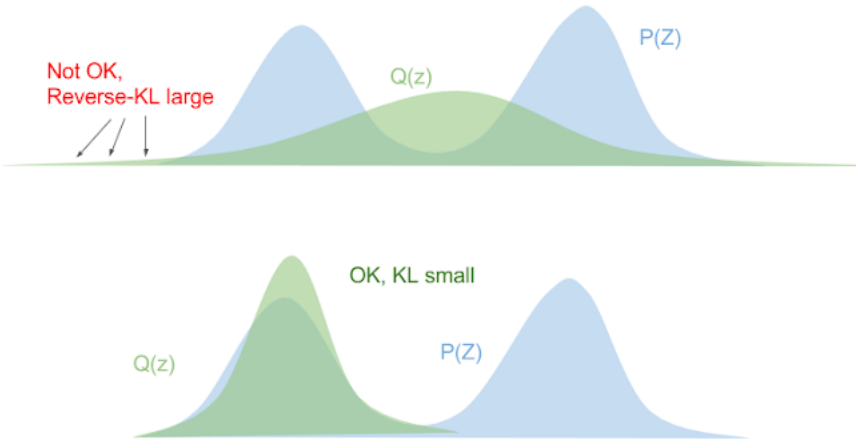


Рис. 18: KL дивергенция между Q и P

0.5.2 Функция потерь

Теперь перейдём непосредственно к обучению. В начале лекции мы говорили с вами про максимизацию функции правдоподобия $\log p(x|\theta)$ для нахождения оптимальных параметров соответствующих данной обучающей выборке. То есть имея обучающие сэмплы $X = \{x_i\}_{i=1}^n$ мы хотим найти такие параметры θ, при которых $p(x|\theta)$ максимально. Так как нам легче работать с суммой, а не с произведением - сразу перейдём к логарифму.

$$\log p(x|\theta) = \int q(z) \log p(x|\theta) dz = \int q(z) \log \frac{p(x,z|\theta)}{p(z|x,\theta)} dz = \int q(z) \log \frac{p(x,z|\theta) q(z)}{p(z|x,\theta) q(z)} dz = \tag{19}$$

$$= \underbrace{\int q(z) \log \frac{q(z)}{p(z|x,\theta)} dz}_{D_{KL}(q(z)|p(z|x)) \geq 0} + \underbrace{\int q(z) \log \frac{p(x,z|\theta)}{q(z)} dz}_{ELBO}$$

Заметим, что логарифм правдоподобия теперь представляется через сумму двух слагаемых:

- KL-дивергенцию распределения латентных переменных и условного распределения латетных переменных при наших данных
- добавки - ELBO (evidence lower bound)

Заметим, что KL-дивергенция у нас неотрицательна и принимает своё минимальное значение в нуле при совпадении почти везде $q(z)$ и $p(z|x)$.

$$\mathcal{L}_{ELBO} = \int q(z) \log \frac{p(x,z)}{q(z)} dz \leq \log p(x) \tag{20}$$

Заметим, что ELBO - это не KL-дивергенция, потому что распределение $p(x,z)$ - совместное распределение x и z, а $p(z)$ - распредедение z.

Таким образом логарифм правдоподобия можно оценить сверху функционалом ELBO (evidence lower bound), которая зависит от параметров энкодера ϕ, кодирующего наше изображение в латентное пространство и параметров декодера θ, декодирующего латент обратно в картинку. Тогда максимизируя ELBO - мы будем максимизировать правдоподобие. Тогда нам нужно максимизировать эту ELBO по параметрам энкодера ϕ, и декодера θ.

$$\mathcal{L}_{ELBO}(\phi, \theta) = \int q(z|\phi) \log \frac{p(x|z,\theta)p(z|\theta))}{q(z|\phi)} dz \to \max_{\theta,\phi} \tag{21}$$

Давайте теперь распишем ELBO.

$$\mathcal{L}_{ELBO}(\phi, \theta) = \int q(z|x,\phi) \log p(x|z,\theta) dz + \int q(z|x,\phi) \log \frac{p(z)}{q(z|x,\phi)} dz = \tag{22}$$

$$\underbrace{\sum_{i=1}^{n} \int q(z_i|x_i,\phi) \log p(x_i|z_i,\theta) dz_i}_{\text{Reconstruction}} - \underbrace{\sum_{i=1}^{n} D_{KL}(q(z_i|x_i,\phi)|p(z_i))}_{\text{Regularization}}$$

Получается две составляющих - **реконструкция** и **регуляризация**. Реконструкционный лосс по сути показывает нам, что будет, если мы возьмём какой-то наш элемент нашей выборки x_i, прогоним его через энкодер, получая распределение с параметрами μ_i, σ_i, засэмплируем из полученного распределения латент z_i, а потом декодируем его обратно в $\hat{x}_i$, насколько результат будет отличаться от исходного сэмпла. Чем более близкое значение будет получаться, тем выше будет реконструкционное слагаемое. А регуляризационный член отвечает за приближение нашим энкодером латентного распределения.

Что было бы, если бы у нас не было регуляризационного слагаемого? Тогда рассмотрим $\log p(x_i|z_i,\theta)$. Нам нужно проинтегрироавть этот логарифм с распределением на z_i. Но этот интеграл будет принимать максимум, если мы просто возьмем дельта функцию в моде. Если у нас q будет иметь какое-то другое распределение, то уже интеграл будте меньше, то есть по сути будет метод максимального правдоподобия, ситуация как у обычного энкодера.

А что будет если у нас останется только второе слагаемое? У -KL дивергенции максимум в нуле и достигается при равенстве распределений, то есть при равенстве априорному распределению.

Посколькую мы хотим в качестве априорного распределения взять нормальное, мы можем переписать KL-дивергенцию для двух нормальных распределений. Давайте для простоты выведем, чему будет равна KL-дивергенция для двух одномерных нормальных распределений, а потом рассмотрим частный случай, когда одно из этих распределений имеет нулевое среднее и единичную дисперсию. Пусть $p_1(x) = \mathcal{N}(x|\mu_1, \sigma_1)$, $p_2(x) = \mathcal{N}(x|\mu_2, \sigma_2)$

$$D_{KL}(p_1 \| p_2) = \int p_1(x) \log \frac{p_1(x)}{p_2(x)} = \underbrace{\int p_1(x) \log p_1(x)}_{A} - \underbrace{\int p_1(x) \log p_2(x)}_{B}$$

$$A = \int \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp^{\frac{-(x-\mu_1)^2}{2\sigma_1^2}} (-\log\sqrt{2\pi\sigma_1} - \frac{(x-\mu_1)}{2\sigma_1} dx =$$

$$= -\log\sqrt{2\pi\sigma_1^2} \underbrace{\int \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp^{\frac{-(x-\mu_1)^2}{2\sigma_1^2}}}_{=1} - \frac{\overbrace{\int p_1(x)x^2 dx}^{=<x^2>} - 2\overbrace{\int p_1(x)\mu_1 x dx}^{=2\mu_1<x>} + \mu_1^2 \overbrace{\int p(x) dx}^{=\mu_1^2}}{2\sigma^2}$$

Заметим, что $< x >= \mu_1$, $\sigma_1^2 =< x^2 > - < x >^2$, тогда

$$A = -\log\sqrt{2\pi\sigma_1^2} - \frac{\sigma_1^2 + \mu_1^2 - 2\mu_1^2 + \mu_1^2}{2\sigma_1^2} = -\frac{1}{2}(1 + \log 2\pi\sigma_1^2)$$

Аналогично разложим слагаемое B:

$$B = -\log\sqrt{2\pi\sigma_2^2} - \frac{\sigma_1^2 + \mu_1^2 - 2\mu_1\mu_2 + \mu_2^2}{2\sigma_2^2}$$

Тогда $A - B = \log\frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$

Вывод для многомерного распределения можно почитать тут.

Тогда если одно из распределений имеет нулевое среднее и единичную дисперсию формула преобразуется следующим образом:

$$D_{KL}(p_1 \| p_2) = \frac{1}{2}(\mu_1 + \sigma_1^2 - 1 - \log\sigma_1^2) \tag{23}$$

0.5.3 Трюк репараметризации

Как мы уже говорили, с помощью энкодера мы предсказываем среднее и дисперсию нормального распределения, однако потом нам необходимо просэмплировать из этого распределения и подать латент в декодер. Такой процесс не является дифференцируемым, а значит мы не можем учить наш автокодировщик. Решением данной проблемы является трюк репараметризации 19.

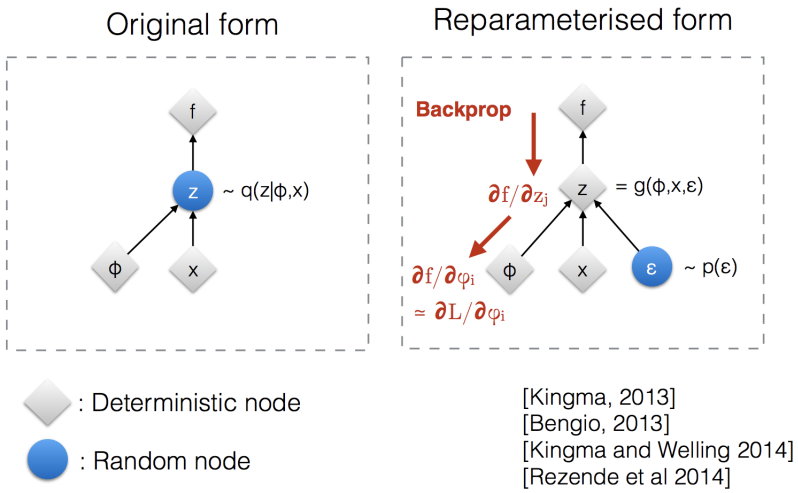


Рис. 19: Трюк репараметризации

Давайте введём дополнительную переменную ϵ - случайная величина. Тогда можно выразить латент как $z = \mu + \sigma \bigodot \epsilon$. В таком случае наш латент обладает всеми теми же свойствами, что и при сэмплировании из нормального распределения, однако теперь можно пропустить градиенты.

0.5.4 Результаты

Давайте посмотрим на результаты реконструкции 20 и генерации 21 изображений из датасесета CelebA [3].

Рис. 20: VAE реконструкция

Рис. 21: VAE генерация

Также с помощью такого автокодировщика мы можем интерполироваться между латентами и редактировать изображения, например, добавлять улыбки, менять цвет волос и много другое 22.

Если посмотреть на результаты, то можно заметить, что вариационный автокодировщик хорошо справляется с быстрой и разнообразной генерацией изображений. Однако результаты получаются не очень качественными, а именно довольно замыленными.

Небольшая затравка на будущее

В идеале наша генеративная модель должна обладать 3 важными свойствами:

1. быстрая генерация
2. разнообразная генерация

Рис. 22: Редактирование изображений с помощью VAE

3. качественная генерация

Также можно рассмотреть три основных типа генеративных моделей:

1. вариационные автокодировщики
2. GaNs
3. диффузионные модели

И стоит важная задача - получить модель, которая может удовлетворить всем трём свойствам 23. Поэтому на данный момент идёт активное исследование ускорение диффузионных моделей, так как на данный момент времени они лучше всего справляются с качественно и разнообразной генерацией.

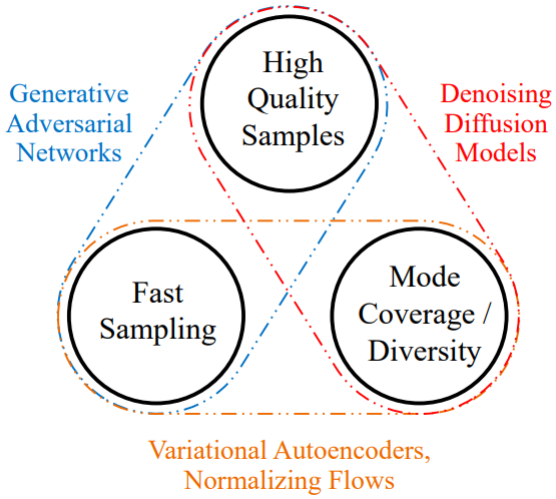


Рис. 23: Генеративная трилемма [7]

0.6 VQVae

Мы рассмотрели вариационный автокодировщик с непрерывным латентным пространством. Но что если это пространство будет дискретным?

Одним из таких автокодировщиков - VQVAE [5]. Алгоритм его работы следующий. Создаём обучаемый словарь эмбеддингов в латентном пространстве $e \in \mathbb{R}^{K \times D}$, где K - количество элементов в словаре, D - размер каждого из векторов. Пусть на вход приходит изображение x, которое прогоняется через энкодер и на выходе получается латентный тензор $z_e(x) \in \mathbb{R}^{W \times H \times D}$, то есть по сути сетка из векторов, имеющих тот же размер, что и векторы в словаре (величины W и H в оригинальной статье берутся равными 32×32, при условии, что само изображение на входе имело размерность $128 \times 128 \times 3$). Далее делается переход к дискретным векторам из нашего словаря по методу ближайших соседей. А далее полученное уже дискретное представление декодируется декодером в изображение.

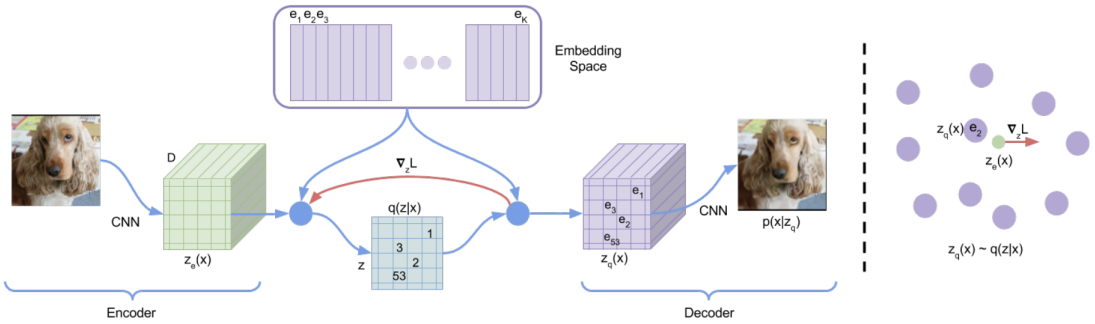


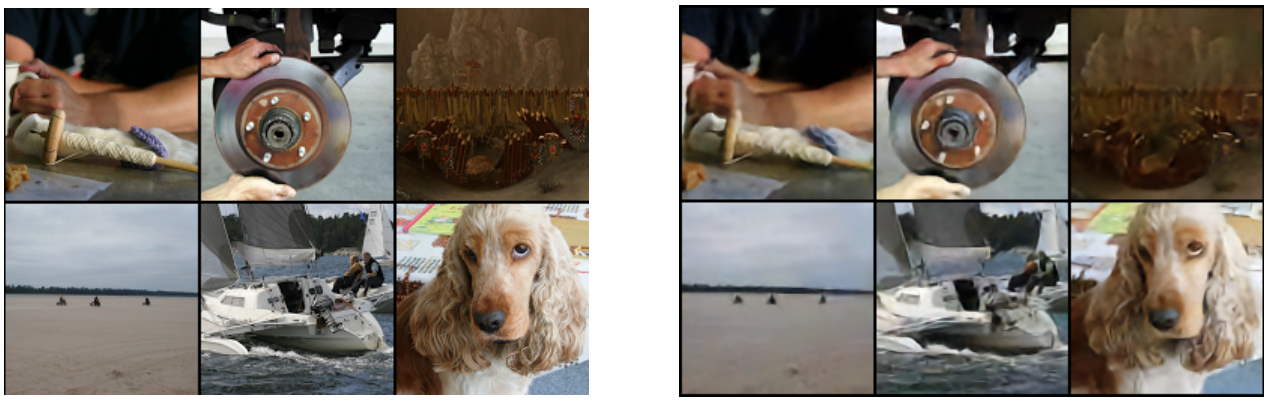
Figure 1: Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2. The gradient $\nabla_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass.

Рис. 24: VQVAE [5]

Теперь возникает вопрос, как это обучать. В данной задаче нам надо обучить энкодер, деркодер, а также словарь векторов. Функцию потерь можно выразить следующим образом:

$$\mathcal{L} = \underbrace{\log p(x|z_q(x))}_{\text{Reconstruction term}} + \|sg[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - sg[e]\|_2^2 \tag{24}$$

Давайте разберёмся с первым слагаемым сначала. По сути это максимизация правдоподобия, о которой говорили раньше, то есть мы по закодированному $z_q(x)$ восстанавливаем x и хотим сделать это как можно точнее. Но возникает проблема – мы не можем пропустить градиент там, где мы просто ищем ближайших соседей. Решается это приёмом, который назвывается **Straight-through gradient estimation**. На прямом проходе мы, как и говорилось ранее, подаём в декодер вектор словаря, найденный через ближайших соседей. На обратном проходе градиент по вектору из нашего словаря $\nabla_z \mathcal{L}$ просто копируется в градиент по вектору, полученному путём кодирования через энкодер.
Вторые два слагаемых в функции потерь просто показывают, что мы хотим приблизить векторы словаря к векторам, получающимся из энкодера. SG - означает стоп градиент, поэтому у нас есть два слагаемых, показывающих, что мы делаем backprop как в параметры энкодера, так и в векторы словаря.
Реконструкция в целом получается неплохая, но немного размыленная 25.



а) б)

Рис. 25: а) Реальные изображения, б) Реконструкция VQVAE [5]

Теперь возникает вопрос, а как хорошо сэмплировать для генерации новых изображений. Для этого можно использовать, например PixelCNN. Что это значит: пусть мы обучили наш энкодер от VQVAE и можем теперь

кодировать изображения. Давайте возьмём теперь и обучим такую сверточную нейросеть, которая по кодам вокруг (уже предсказанным) может предсказывать код (распределение вероятностей кодов) в данном пикселе 26.

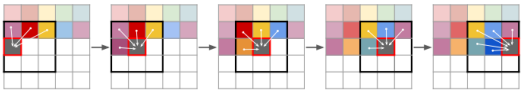
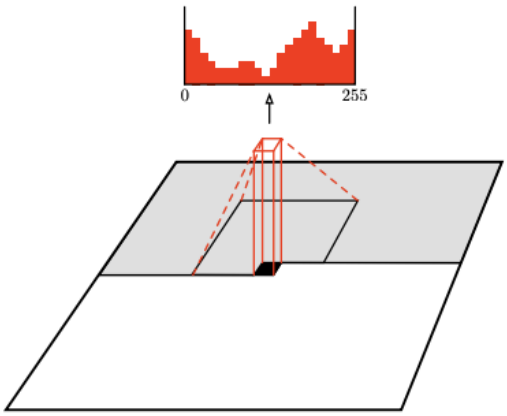


Figure 3. Sliding attention window.

[2]



[6]

Рис. 26: Основная идея PixelCNN

Таким образом, мы можем сгенерировать латентный тензор кодов, который затем подать в наш декодер VQVAE и предсказать непосредственно само изображение. Результаты можно посмотреть на рисунке 27.

admiral (butterfly) alp brown bear tiger beetle kitfox

Рис. 27: Генерация VQVAE [5]

0.7 Выводы

Таким образом мы с вами узнали, что такое генеративная модель, познакомились с идеей автокодировщика и какие задачи он может решать, сравнили как работает вариационный автокодировщик и автокодекодер и наконец узнали, что вариационный автокодировщик может также работать с дискретным латентным пространством.

Список литературы

[1] Piotr Bojanowski и др. “Optimizing the latent space of generative networks”. В: *arXiv preprint arXiv:1707.05776* (2017).

[2] Patrick Esser, Robin Rombach и Bjorn Ommer. “Taming transformers for high-resolution image synthesis”. В: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, с. 12873—12883.

[3] Ziwei Liu и др. “Deep Learning Face Attributes in the Wild”. В: *Proceedings of International Conference on Computer Vision (ICCV)*. Дек. 2015.

[4] Xiao-Jiao Mao, Chunhua Shen и Yu-Bin Yang. “Image restoration using convolutional auto-encoders with symmetric skip connections”. В: *arXiv preprint arXiv:1606.08921* (2016).

[5] Aaron Van Den Oord, Oriol Vinyals и др. “Neural discrete representation learning”. В: *Advances in neural information processing systems* 30 (2017).

[6] Aäron Van Den Oord, Nal Kalchbrenner и Koray Kavukcuoglu. “Pixel recurrent neural networks”. В: *International conference on machine learning*. PMLR. 2016, с. 1747—1756.

[7] Zhisheng Xiao, Karsten Kreis и Arash Vahdat. “Tackling the generative learning trilemma with denoising diffusion gans”. В: *arXiv preprint arXiv:2112.07804* (2021).